# 什么是版本控制系统（VCS）

很多人认为 Git 难以理解的第一个门槛在于：所谓的「Git 是一个分布式版本控制系统」这句话的具体含义不够清楚。其实分布式版本控制系统（Distributed Version Control System - DVCS）这个定义并不难，不过一步一步来，我先告诉你，什么是版本控制系统（Version Control System - VCS）。

## 版本控制：最基本功能

> 版本控制系统（VCS）最基本的功能是版本控制。所谓版本控制，意思就是在文件的修改历程中保留修改历史，让你可以方便地撤销之前对文件的修改操作。

最简化的版本控制模型，是大多数主流文本编辑器都有的「撤销（Undo）」功能：你本来想删除一个字符，却在按删除键之前不小心选中了全文，结果一下子整篇文档都被删光了，没关系，按一下「撤销」(Ctrl + Z 或 ⌘ + Z 或 U 等等，具体和你的操作系统以及编辑器有关），删掉的文字就都回来了。这其实是文本编辑器帮你自动保存了之前的内容，当你按下「撤销」的时候，它就帮你把内容回退到上一个状态；同理，按一次是会退到上一个版本，按两次就是回退到上上一个版本。

写程序的时候同样也难免会遇到「写错」的情况，所以程序的 VCS，当然也会需要版本控制功能，这样当你发现「昨天有一行代码写错了」，你就不用凭着记忆把那段代码背出来，而只需要在 VCS 中选择撤回到昨天的那个版本。

## 主动提交：程序代码和普通文本的区别

VCS 和文本编辑器的撤销功能比起来，有一个很重要的区别是：程序代码的修改的生命周期非常长。一次代码的修改，在几天后、几个月后、几年后都有可能需要被翻出来。如果依然采用「每次改动自动保存」的形式来保留修改历史，将会导致改动历史非常频繁和无章可循，这样，历史代码的查找、阅读和回退就会很困难了。所以，和文本编辑器的撤销功能不同，VCS 保存修改历史，使用的是**主动提交改动**的机制。

在你写了一段完整的代码（例如修复了一个 bug）之后，使用 commit 命令把改动和对改动的描述信息提交，这次改动就被记录到版本历史中了。之后如果你希望回退到这个版本，就可以从 VCS 的历史日志中方便地找到它。

### 多人合作的同步需求：中央仓库

代码可以一个人写，但更多的时候会是多个人共同开发。那么自然地，就需要有一个中央仓库作为代码的存储中心：所有人的改动都会上传到这里，所有人都能也都能看到和下载到别人上传的改动。

这样，解决了同步的需求，多个人在不同的机器上开发同一个程序就成了可能。

**版本控制**、**主动提交**、**中央仓库**这三个要素，共同构成了版本控制系统（VCS）的核心：开发团队中的每个人向中央仓库主动提交自己的改动和同步别人的改动，并在需要的时候查看和操作历史版本，这就是版本控制系统。

## 中央式版本控制系统

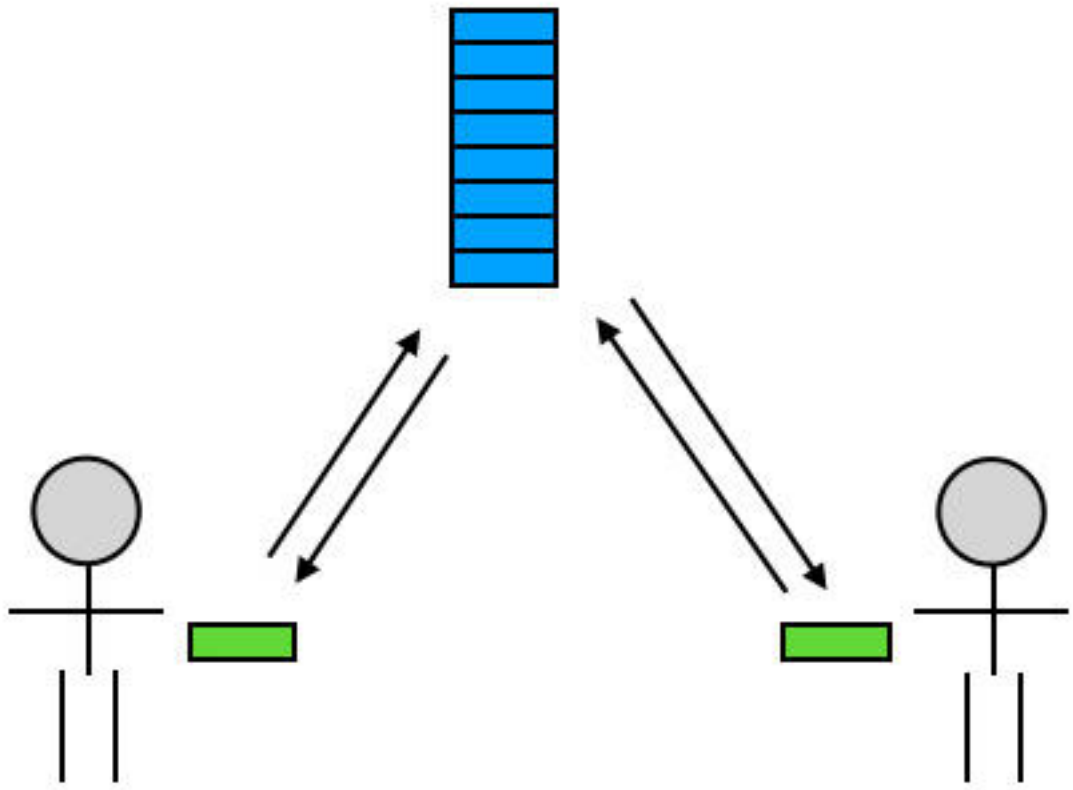
最初的版本控制系统，是中央式版本控制系统（Centralized VCS），也就是前面我讲的这种。Git 是分布式的版本控制系统（Distributed VCS），它和中央式的区别我在下节说，现在先说一下中央式版本控制系统的工作模型。

### 工作模型

假设你在一个三人团队，你们计划开发一个软件或者系统，并决定使用中央式 VCS 来管理代码。于是：

1. 作为项目的主工程师，你独自一人花两天时间搭建了项目的框架；
2. 然后，你在公司的服务器（这个服务器可以是公司内的设备，也可以是你们买的云服务）上**创建了一个中央仓库，并把你的代码提交到了中央仓库上**；
3. 你的两个队友**从中央仓库取到了你的初始代码**，从此刻开始，你们三人开始**并行开发**；

4. 在之后的开发过程中，你们三人为了工作方便，总是每人独立负责开发一个功能，在这个功能开发完成后，这个人就把他的这些**新代码提交到中央仓库**；
5. 每次当有人把代码提交到中央仓库的时候，另外两个人就可以选择**把这些代码同步到自己的机器上**，保持自己的本地代码总是最新的。



中央式 VCS

而对于团队中的每个人来说，就会更简单一点：

1. 第一次加入团队时，把中央仓库的代码取下来；
2. 写完的新功能提交到中央仓库；
3. 同事提交到中央仓库的新代码，及时同步下来。

这样，一个三人的团队就成功做到了各自在自己的电脑上开发同一个项目，并且互不影响，就好像你们三个人是在同一台电脑上操作一样。

这就是中央式 VCS 最基本的工作模型。当然，实际的开发工作并没有简单到这种程度，因为你时常会需要处理代码冲突、查看版本历史、回退代码版本等；另外，Git 属于分布式 VCS，它的概念也比中央式 VCS 要复杂一些。但这些概念你需要一步步地理解和吸收，你现在只需要先知道中央式 VCS 的这个基本工作模型，其他的内容我会在后面慢慢地全部讲清楚。

# 什么是分布式版本控制系统（DVCS）

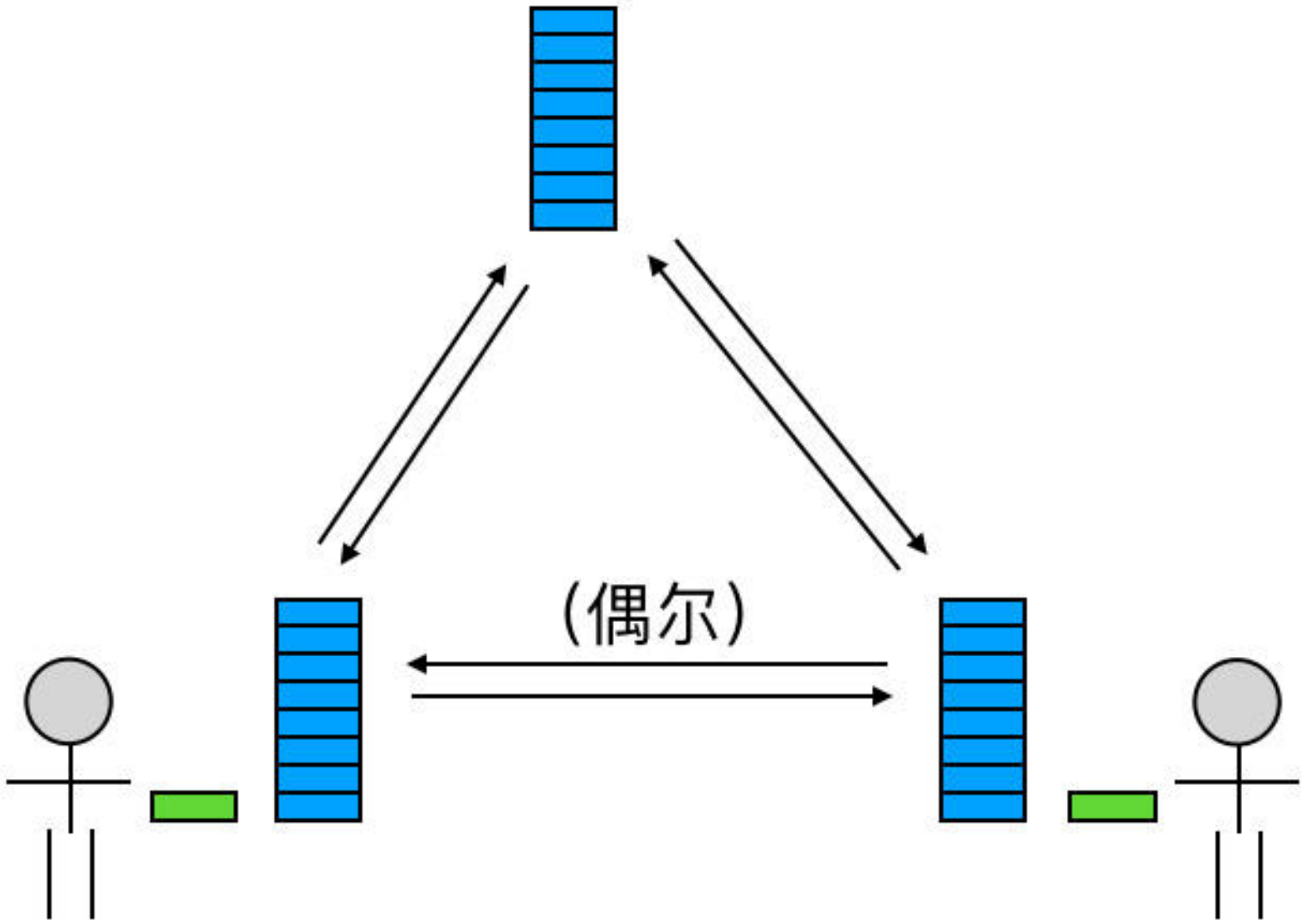
分布式 VCS （Distributed VCS / DVCS）和中央式的区别在于，分布式 VCS 除了中央仓库之外，还有本地仓库：团队中每一个成员的机器上都有一份本地仓库，这个仓库里包含了所有的版本历史，或者换句话说，每个人在自己的机器上就可以提交代码、查看历史，而无需联网和中央仓库交互——当然，取而代之的，你需要和本地仓库交互。

中央式 VCS 的中央仓库有两个主要功能：**保存版本历史**、**同步团队代码**。而在分布式 VCS 中，保存版本历史的工作转交到了每个团队成员的本地仓库中，中央仓库就只剩下了同步团队代码这一个主要任务。它的中央仓库依然也保存了历史版本，但这份历史版本更多的是作为团队间的同步中转站。

## 工作模型

依然以三人团队为例，分布式 VCS 的工作模型大致是这样：

1. 首先，你作为主工程师，独立搭建了项目架构，**并把这些代码提交到了本地仓库**；
2. 然后，你在服务器上创建了一个中央仓库，并把 1 中的提交**从本地仓库推送到了服务器的中央仓库**；
3. 其他同事**把中央仓库的所有内容克隆到本地，拥有了各自的本地仓库**，从此刻开始，你们三人开始并行开发；
4. 在之后的开发过程中，你们三人总是每人独立负责开发一个功能，在这个功能开发过程中，**一个人会把它的每一步改动提交到本地仓库**。注意：由于本地提交无需立即上传到中央仓库，所以每一步提交不必是一个完整功能，而可以是功能中的一个步骤或块。
5. 在一个人把某个功能开发完成之后，他就可以把这个功能相关的所有提交**从本地仓库推送到中央仓库**；
6. 每次当有人把新的提交推送到中央仓库的时候，另外两个人就可以选择**把这些提交同步到自己的机器上，并把它们和自己的本地代码合并**。




分布式 VCS

可以看出，这个工作模型和上一节讲的「中央式 VCS 的工作模型」很相似，只是把代码的提交和上传过程拆开了。

另外，和上节讲的中央式 VCS 工作模型一样，这个也只是分布式 VCS 的一个最基本的工作模型，实际的开发工作会比这个麻烦和复杂。但这是个核心模型，你把它理解了，就可以更好地看懂后面的内容。

## 优点与缺点

分布式 VCS 的优点：

1. 大多数的操作可以在本地进行，所以速度更快，而且由于无需联网，所以即使不在公司甚至没有在联网，你也可以提交代码、查看历史，从而极大地减小了开发者的网络条件和物理位置的限制（例如，你可以在飞机上提交代码、切换分支等等）；
2. 由于可以提交到本地，所以你可以分步提交代码，把代码提交做得更细，而不是一个提交包含很多代码，难以 review 也难以回溯。

分布式 VCS 的缺点：

1. 由于每一个机器都有完整的本地仓库，所以初次获取项目（Git 术语：clone）的时候会比较耗时；
2. 由于每个机器都有完整的本地仓库，所以本地占用的存储比中央式 VCS 要高。

> 对于一般的程序项目而言，由于项目的大多数内容都是文本形式的代码，所以工程的体积都并不是很大，再加上文本内容自身的特点，VCS 可以利用算法来把仓库的体积极大地压缩。这就导致，在实际中，Git 等分布式 VCS 的仓库体积并不大，初次获取项目的耗时和本地仓库的存储占用都很小。所以对于大多数的程序项目而言，分布式 VCS 「尺寸大、初次下载慢」的问题其实并不严重。
>
> 不过也有一些例外，比如游戏开发。游戏的开发中有大量的大尺寸数据和媒体文件，并且这些文件的格式也不容易压缩尺寸，如果用分布式 VCS 会导致仓库的体积非常庞大。所以一些大型游戏的开发会选择中央式的 VCS 来管理代码。

# 上手 1：新公司用 Git 管理代码，怎么快速上手？

> 已经有 Git 经验的可以跳过这一节

刚进入新公司，被告知团队是用 Git 管理项目代码的，而你却从来没用过 Git。纵然同事告诉你「没事，先自学一下」「有问题可以问我」，但你肯定不想一进公司就花太多时间在自我学习上，也不想过多地打扰这些刚认识的同事。怎么办？

对你来说，最重要的是，**先知道 Git 怎么用**。先把最基本的掌握了，至于正规团队使用 Git 有什么额外要求、Git 有什么高级用法、Git 的原理这些，都可以放在这之后。万事开头难，你先把 Git 最基本的使用掌握了，后面的再一步一步来。

### 安装 Git

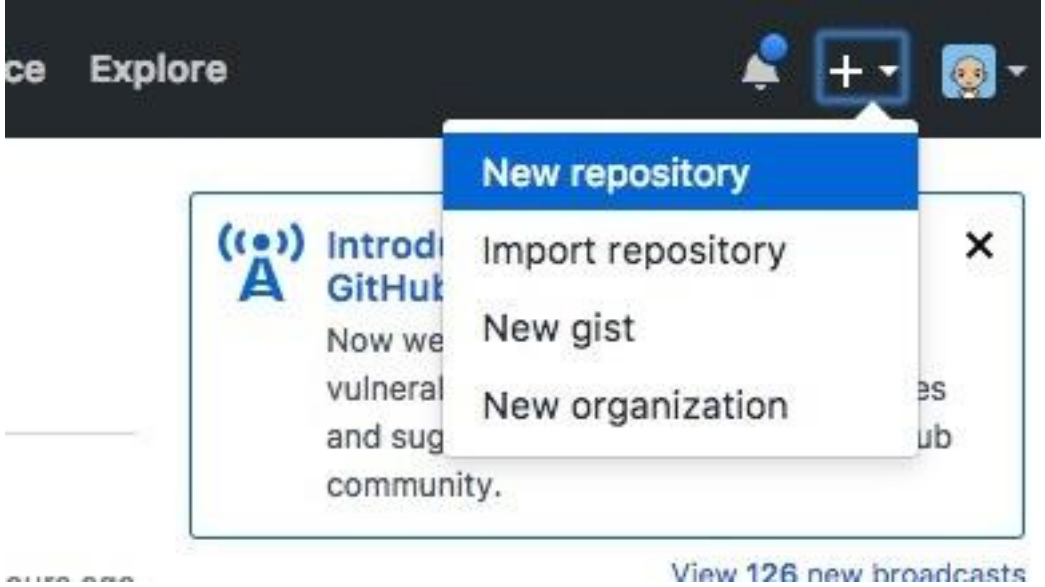
点击这里去下载个 Git，安装到你的机器上。或者如果你喜欢用 Homebrew 或 apt 什么的来安装都好，总之，把它安装好。

装好以后，就可以正式开始上手 Git 了。

## 先建个练习项目

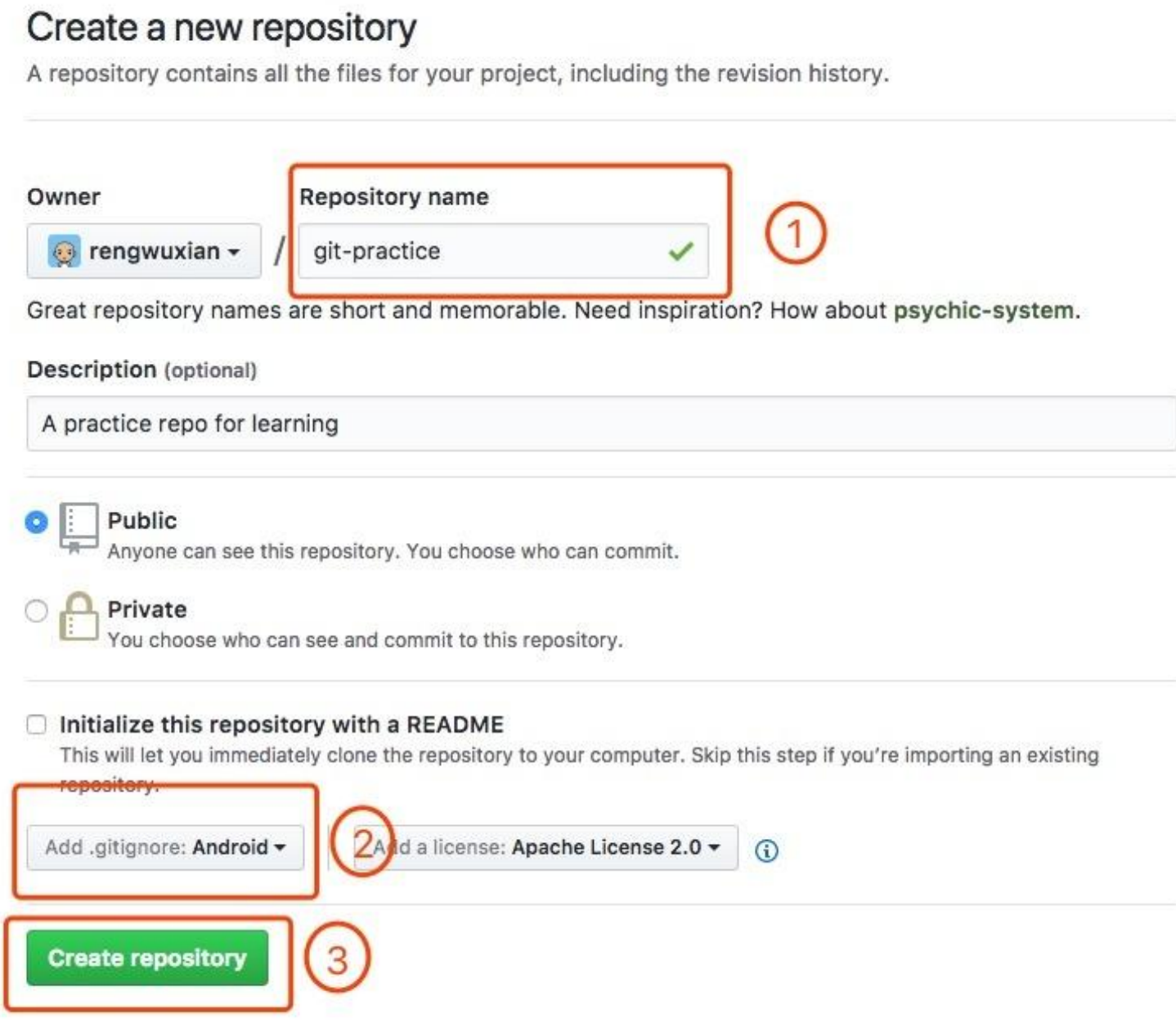
学习的时候最好别拿团队的正式项目练手，先在 GitHub 上建一个自己的练习项目。

1. 访问 GitHub（用别的平台比如 bitbucket 什么的也行）
2. 注册或登录您的账号
3. 点击右上角的「New Repository」来新建远程仓库



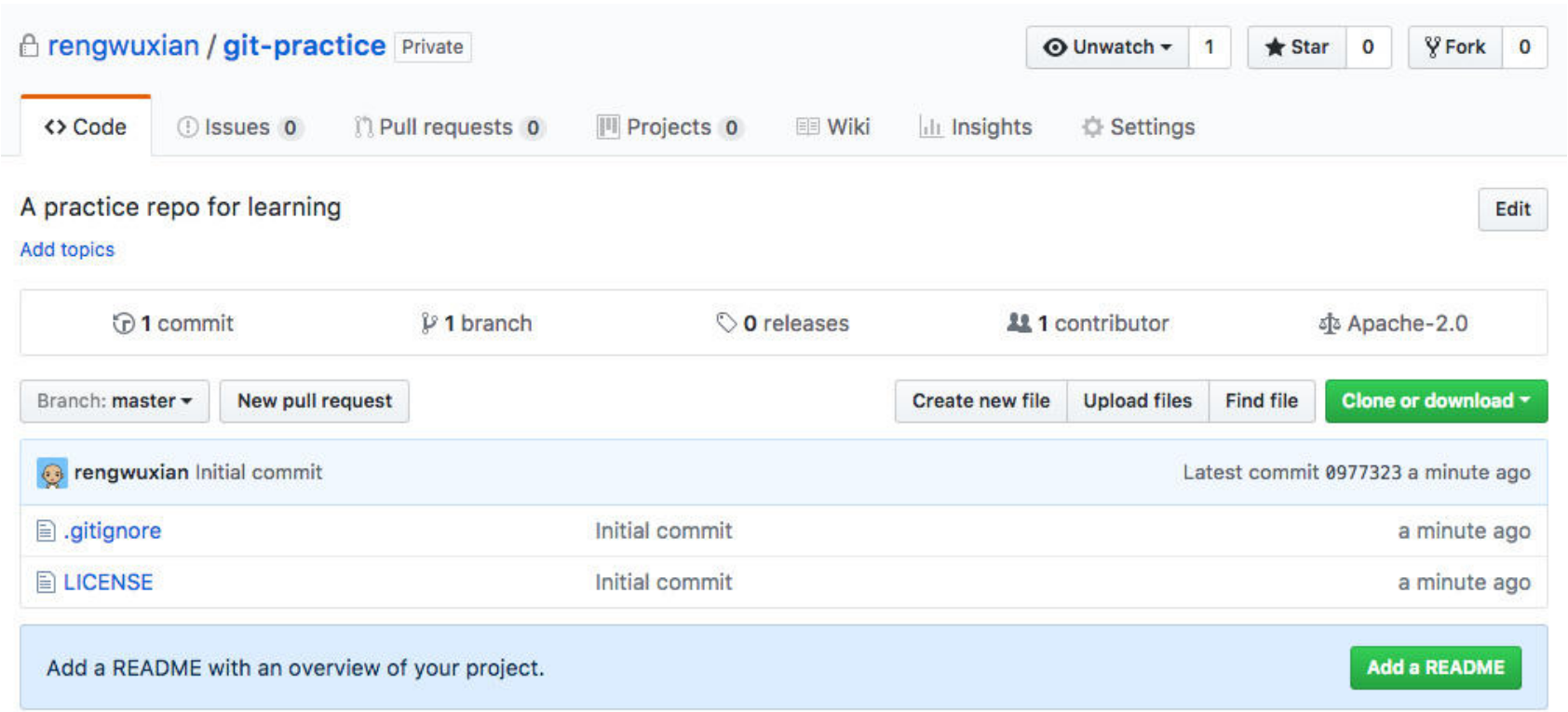

建立一个新的仓库

4. 进入仓库设置页面填写信息：其中 ① 是你的仓库名，这个仓库名同样会被 GitHub 设置为你的仓库的根目录的名称；② 是为 `.gitignore` 设置项目类型，`.gitignore` 是 Git 仓库中的一个特殊的文本文件，它里面记录了你不希望提交到仓库的目录和文件的名称或类型，例如你的 `/build` 目录；把 ① 和 ② 填好之后，就可以点 ③ 来完成远程仓库的创建了



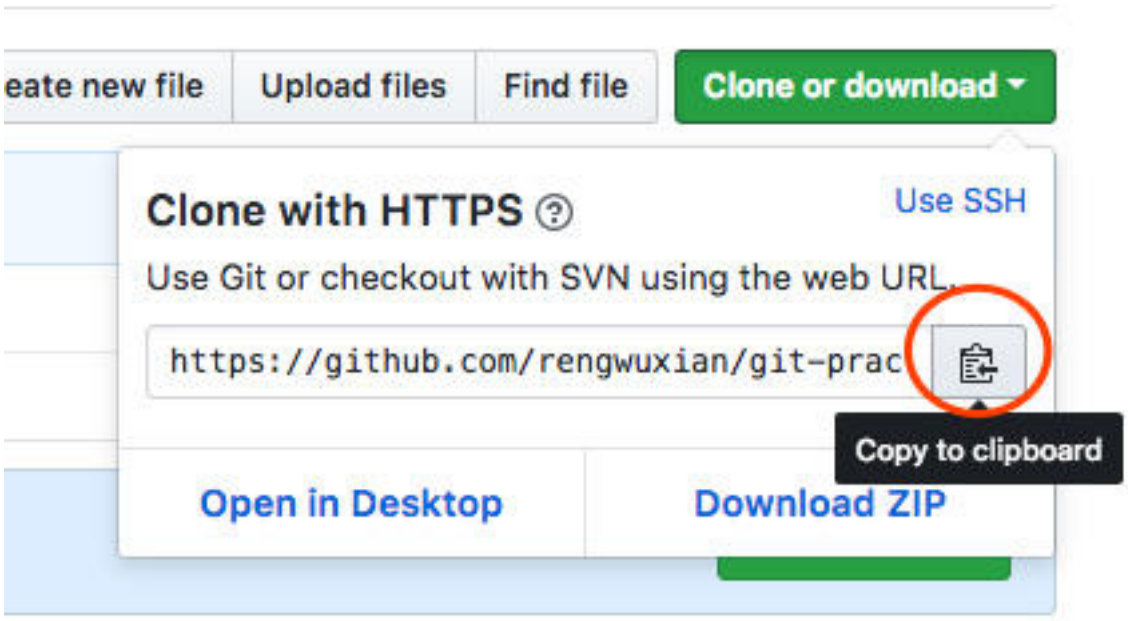
创建仓库界面

创建完的远程仓库大概长这样：




刚创建好的 GitHub 仓库

点击右边的「Clone or download」，然后把仓库的 `clone` 地址复制到剪贴板：




复制仓库的 Clone 链接

> clone 地址是什么？下面马上就说啦！

## 把远程仓库取到本地

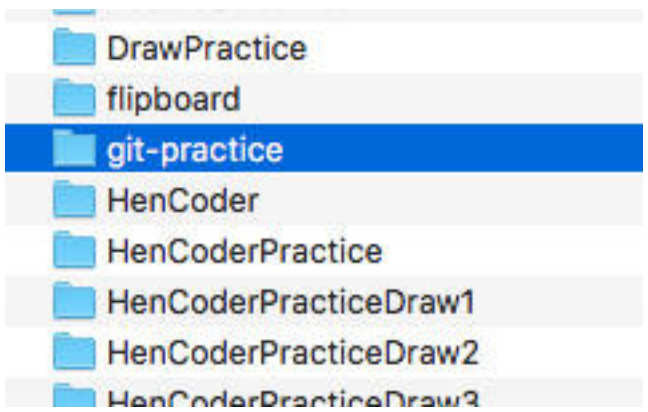
接下来就可以把远程仓库取下来了。取的方式很简单：在 Terminal 或 cmd 中切换到你希望放置项目的目录中，然后输入：

```
git clone 你刚复制的地址
```

Git 就会把你的远程仓库 clone 到本地。在这个过程中，你可能会需要输入你的 GitHub 用户名和密码：

用 git clone 把仓库下载到本地

输入正确的用户名和密码以后，你会看到你的当前目录下多了一个新的子目录，它的名字和刚才新建的 GitHub 仓库名一致：




进入这个目录，你会发现这里除了你刚才添加的 `LICENSE` 和 `.gitignore` 文件外，还有一个叫做 `.git` 的隐藏目录。

这个 `.git` 目录，就是你的**本地仓库（Local Repository）**，你的所有版本信息都会存在这里。而 `.git` 所在的这个根目录，称为 Git 的**工作目录（Working Directory）**，它保存了你当前从仓库中签出（checkout）的内容。现在你在项目的目录下输入：

```
git log
```

```
commit 0977323580be59378cf84fea791bb2e87ef21411 (HEAD -> master, origin/master,
origin/HEAD)
Author: Kai Zhu <rengwuxian@gmail.com>
Date:   Sun Nov 19 09:51:11 2017 +0800

    Initial commit
(END)
```

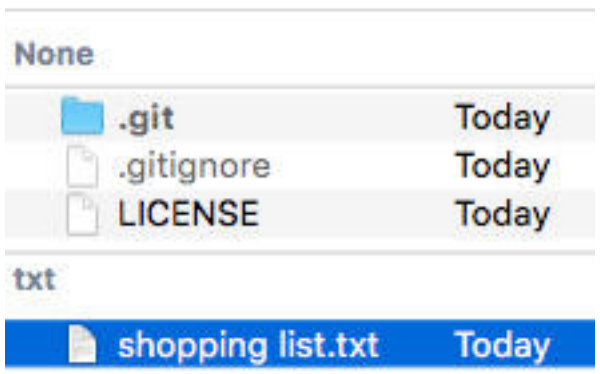
仓库的提交历史

在这里你只能看到一个提交，这个提交是 GitHub 帮你做的，它的内容是创建你的初始 `.gitignore` 和 `LICENSE` 这两个文件。图中第一行中的 `commit` 右边的那一大串字符（09773235...21411），是这个 `commit` 的 SHA-1 校验和（如果不知道什么是 SHA-1，你可以暂时把它简单理解为这个 `commit` 的 ID）；后面括号里的内容（HEAD -> master ...）稍后再讲；第一行的下面，依次是这个 `commit` 的作者、提交日期和提交信息，其中提交信息记录了这个提交做了什么，是提交者填写的（当然，这条提交信息是 GitHub 帮你写的）。

简单看一下这些信息，然后按 `q` 键退出吧，往下继续。

## 自己写个提交试试

把远程仓库取到本地之后，你就可以开始尝试提交代码了。不过为了方便，你不必真的写代码，你可以在工作目录下创建一个文本文件，例如 `shopping list.txt`。



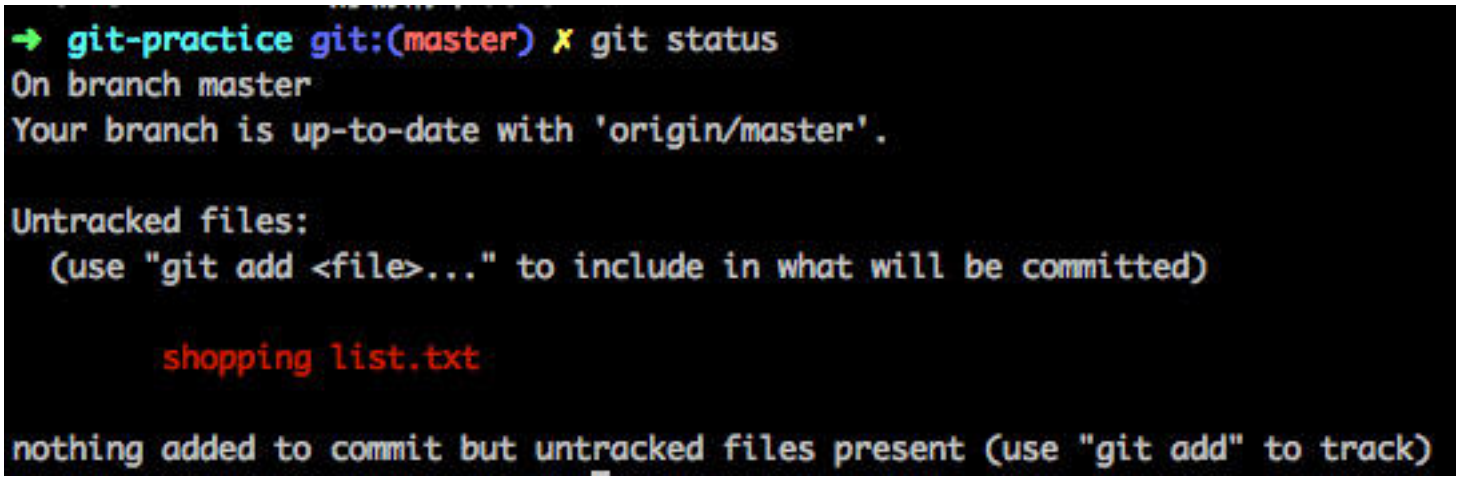

添加文件 shopping list.txt

shopping list.txt 中的内容

Git 的提交是用的 `commit` 指令。不过.....你现在还不能直接提交。你现在在 Terminal 输入：

```
git status
```

`status` 是用来查看工作目录当前状态的指令：




输入 git status 后的输出画面

这段文字表述了很多项信息：

1. 你在 `master` branch
2. 当前 branch 没有落后于 `origin/master`
3. 你有 untracked files （未追踪的文件），文件名是 `shopping list.txt`。

4. 你可以使用 `git add` 来开始追踪文件。

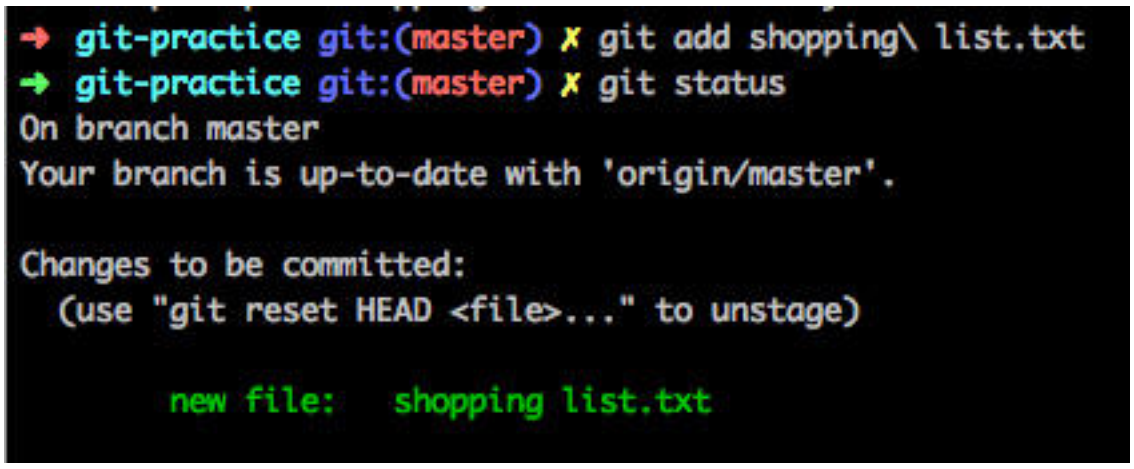
其中前两条你可以暂时先不理，branch 的东西我在后面会讲。关于后两条，简单说一下：

从上面的信息可以看出，`shopping list.txt` 这个文件目前属于 "untracked" 状态，它的意思是 Git 仓库对它没有进行任何记录，你在提交的时候不会把它提交上去，查看提交历史也不会看到它。总之，对于 Git 仓库来说，它是不存在的。

而你现在想提交这个文件，所以首先，你需要用 `add` 指令来让 Git 开始跟踪它：

```
git add shopping\ list.txt
```

输入这行代码，Terminal 不会给你反馈信息。但这时你再执行一次 `git status`，你会发现显示内容变了：




可以看到，`shopping list.txt` 的文字变成了绿色，它的前面多了「new file:」的标记，而它的描述也从 "Untracked files" 变成了 "Changes to be commited"。这些都说明一点：`shopping list.txt` 这个文件的状态从 "untracked"（未跟踪）变成了 "staged"（已暂存），意思是这个文件中被改动的部分（也就是这整个文件啦）被记录进了 staging area（暂存区）。

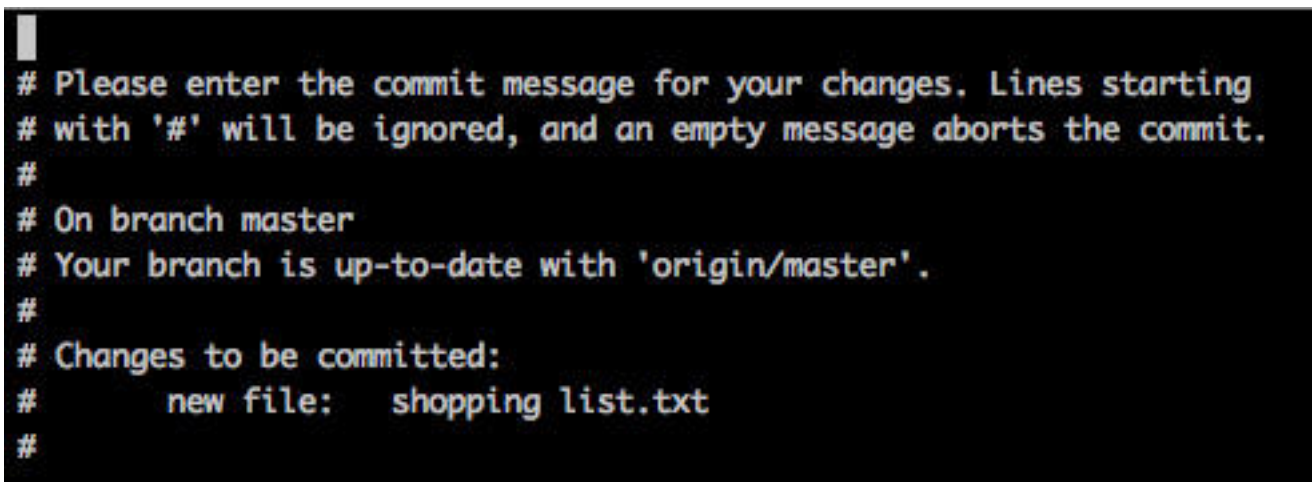
> 解释一下 "stage" 这个词，这个词对我们中国人可能有一些理解难度。按我们英语课本上的内容来看，stage 是一个名词，它的意思是「舞台」。可是不论从词性还是词义，「舞台」都不太能解释 "stage" 或

> "staging area" 的意思。实质上，Git 中的 stage 取自这个词的另一个意思：组织和准备某个事件。而 "staging area" 的意思也并不是「舞台区域」，而是「用来汇集物品或材料以备使用的区域」的意思。
>
> 所以 stage 这个词在 Git 里，是「集中收集改动以待提交」的意思；而 staging area，就是一个「汇集待提交的文件改动的地方」。简称「暂存」和「暂存区」。至于 staged 表示「已暂存」，就不用再解释了吧?

所谓的 staging area，是 `.git` 目录下一个叫做 `index` 的文件（嗯，它的文件名并不叫 `stage`）。你通过 `add` 指令暂存的内容，都会被写进这个文件里。

现在文件已经放进了暂存区，就可以提交了。提交的方式是用 `commit` 指令：

```
git commit
```




然后你会进入这样一个界面，这个界面是用来填写提交信息（commit message）的。根据操作系统以及设置的不同，这个界面的编辑器可能是 nano 或者 vi 或者别的什么，总之如果你不会用它，那么建议你尽快上网搜搜它的用法，因为 Git 的操作会经常用到它。在这里我只简单说一下它的最基本用法：

- 在初始状态下，你是在命令模式，不能编辑这个文件，你需要按一下 "i"（小写）来切换到插入模式，然后就可以输入你的提交信息了：

```
Add "shopping list.txt"
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
# Your branch is up-to-date with 'origin/master'.
#
# Changes to be committed:
#       new file:   shopping list.txt
#
```

- 在输入完成后别按回车，而是要按 ESC 键返回到命令模式，然后连续输入两个大写的 "Z"（用 Shift 键或 Capslock 键都可以），就保存并退出了。

```
➜ git-practice git:(master) ✗ git commit
[master 79c353d] Add "shopping list.txt"
 1 file changed, 10 insertions(+)
 create mode 100644 shopping list.txt
```

这样，一次提交就完成了。如上图这样，从界面中你可以看到这次提交的简单信息。这时如果你再执行一次刚才执行过的 `git log`（还记得这个指令是什么意思吗？它会列出你的提交历史）：

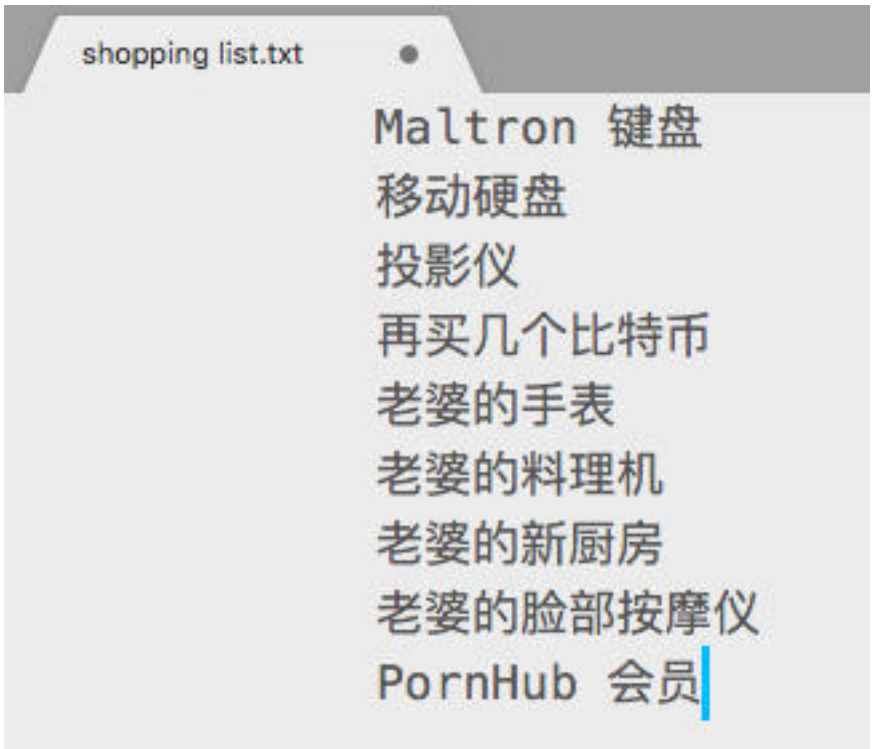
```
git log
```

```
commit 79c353da8263fd9e93d7eb68dfbd1c8190d3450c (HEAD -> master)
Author: Kai Zhu <rengwuxian@gmail.com>
Date:   Sun Nov 19 14:31:54 2017 +0800

    Add "shopping list.txt"

commit 0977323580be59378cf84fea791bb2e87ef21411 (origin/master, origin/HEAD)
Author: Kai Zhu <rengwuxian@gmail.com>
Date:   Sun Nov 19 09:51:11 2017 +0800

    Initial commit
(END)
```

可以看到，你的这条提交被列在了最上面，现在你的提交历史中有两条记录了。这说明，你已经成功做了一次提交到本地仓库，它已经被保存在了 `.git` 这个目录里的某个地方了。

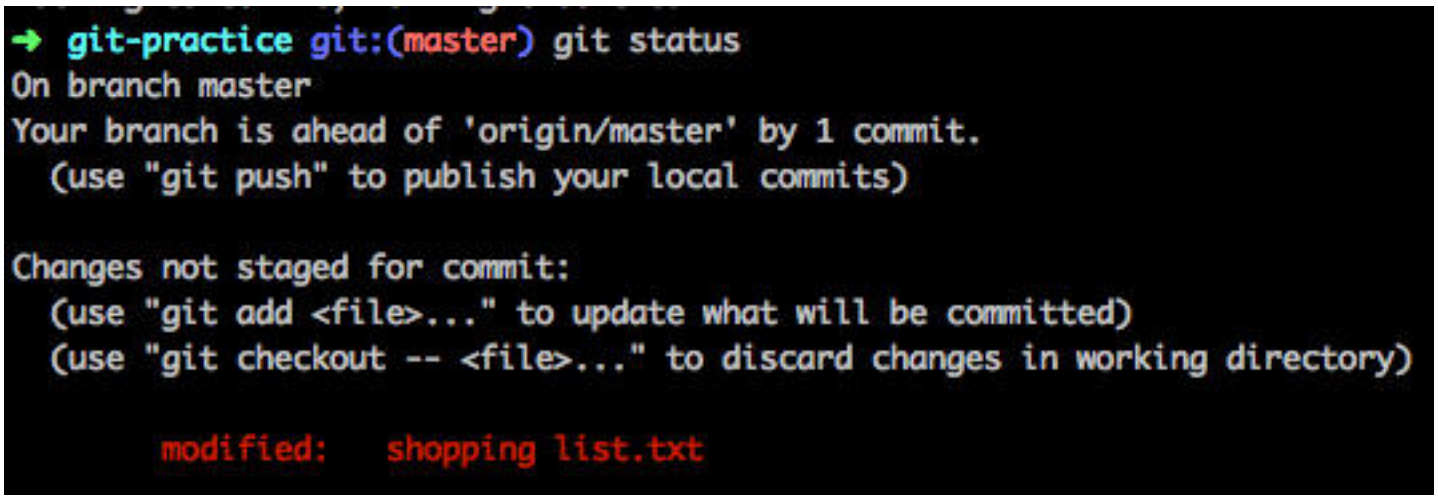
## 再来个提交

想来想去，我还是觉得把「女装」列在购物清单有点太羞耻了，所以还是把它删掉吧：




嗯删掉以后脸不烫了，赶紧把它提交上去。提交之前先看看文件状态是个好习惯：

```
git status
```




可以看到，`shopping list.txt` 又变红了，不过这次它左边的文字不是 "New file:" 而是 "modified:"，而且上方显示它的状态也不是 "Untracked" 而是 "not staged for commit"，意思很明确：Git 已经认识这个文件了，它不是个新文件，但它有了一些改动。所以虽然状态的显示有点不同，但处理方式还是一样的：

```
git add shopping\ list.txt
```

这时再 status 一下，就会看到 shopping list.txt 已经 staged ("to be commited") 了：

```
git status
```

```
➜ git-practice git:(master) ✗ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        modified:   shopping list.txt
```

好，最后一步，commit：

```
git commit
```

```
Remove a shameful part
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch master
# Your branch is ahead of 'origin/master' by 1 commit.
#   (use "git push" to publish your local commits)
#
# Changes to be committed:
#       modified:   shopping list.txt
#
```

这时再看看 log，你已经有三条 commits 了：

```
git log
```

```
commit 5e68b0d8d47ebd2a52977586f728f74335ae62c5 (HEAD -> master)
Author: Kai Zhu <rengwuxian@gmail.com>
Date:   Sun Nov 19 14:58:33 2017 +0800

    Remove a shameful part

commit 79c353da8263fd9e93d7eb68dfbd1c8190d3450c
Author: Kai Zhu <rengwuxian@gmail.com>
Date:   Sun Nov 19 14:31:54 2017 +0800

    Add "shopping list.txt"

commit 0977323580be59378cf84fea791bb2e87ef21411 (origin/master, origin/HEAD)
Author: Kai Zhu <rengwuxian@gmail.com>
Date:   Sun Nov 19 09:51:11 2017 +0800

    Initial commit
(END)
```

## 把提交推送到中央仓库

到现在为止，已经写了两条提交，但它们都还在本地仓库。为了把代码分享出去，你还需要把这些提交上传到中央仓库。如果你现在再看一下 `status`：

```
git status
```

```
➜ git-practice git:(master) git status
On branch master
Your branch is ahead of 'origin/master' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

你会看到，Git 提示你的当前 branch 已经领先于（"ahead of"）'origin/master' 两个提交了。就像上面我说的，branch 这个概念我到后面再讲，`origin/master` 的中的 `origin` 是远端仓库的名称，是你在用 `clone` 指令初始化本地仓库时 Git 自动帮你起的默认名称；`master` 是 `origin` 上的分支名称。不过对于现在来说，可以暂时把 `origin/master` 简单理解为「中央仓库」。也就是说，这句话是告诉你，你的本地仓库已经领先中央仓库两个提交了。这两个提交很明显就是指的刚才那两个关于 `shopping list.txt` 的提交。

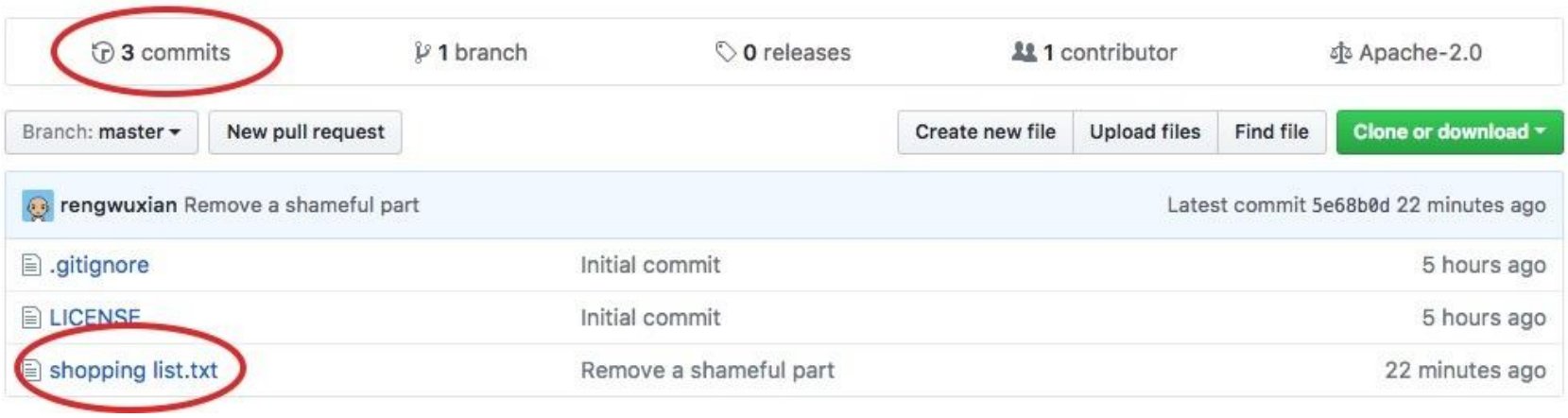
而这句话的下面也说明了，你可以使用 `git push` 来把你的本地提交发布（即上传到中央仓库）。所以很简单，照做吧：

```
git push
```

```
➜ git-practice git:(master) git push
Username for 'https://github.com': rengwuxian
Password for 'https://rengwuxian@github.com':
Counting objects: 6, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 697 bytes | 697.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/rengwuxian/git-practice.git
   0977323..5e68b0d  master -> master
```

提交本地仓库到远程服务器

由于这是联网操作，所以在这个过程 GitHub 会再次向你索要账户和密码。填入正确的账户和密码，`push` 操作就完成了。这时你再去你的 GitHub 仓库页面看一下：




GitHub 的仓库页面可以看到相应的更新

你会发现 `shopping list.txt` 文件已经在上面了，并且 `commits` 的数量从一个变成了 3 个。这说明你已经成功把本地仓库的提交推送到了服务器。

> 如果觉得一遍遍地输入密码很烦，可以按照这个页面 提供的方案来把密码保存起来。
>
> 另外还有一个更简单但安全性低一些的方案。执行这行代码：

```
git config credential.helper store
```

> 在这之后你只需要再输入一次密码， Git 就会把你的密码保存下来，这之后就再也不用输入了。说它「安全性低」，是因为这条指令会让 Git 把你的密码以明文形式保存在你的电脑上。具体这两种保存密码的方案选择哪个，看你自己了。

## 小结

在这一节里，描述了 Git 的最基本的工作模型：

1. 从 GitHub 把中央仓库 clone 到本地（使用命令： `git clone`）
2. 把写完的代码提交（先用 `git add 文件名` 把文件添加到暂存区，再用 `git commit` 提交）
    - 在这个过程中，可以使用 `git status` 来随时查看工作目录的状态
    - 每个文件有 "changed / unstaged"（已修改）, "staged"（已修改并暂存）, "commited"（已提交） 三种状态，以及一种特殊状态 "untracked"（未跟踪）
3. 提交一次或多次之后，把本地提交 `push` 到中央仓库（`git push`）

下一节将会讲一下在团队开发中会遇到的问题，从而以这种工作模型为基础，延伸到团队开发的基本工作模型。

但在这之前，强烈建议你先按照这节的内容照做一遍，并尝试理解这一节中所说的工作模型。不然的话，可能会出现「小册看完了，却还是不知道怎么做」的情况。

# 上手 2：团队工作的基本工作模型

对于 Git 来说，团队合作和个人独立工作最大的不同在于，你会提交代码，别人也会提交；你会 push，别人也会 push，因此除了把代码上传，每个人还需要把别人最新的代码下载到自己的电脑。而且，这些工作都是并行进行的。

这就很复杂了！

这节的内容，就是简单阐述一下团队合作中 Git 的基本工作模型。

## 把别人的新提交拿到本地

先说一种简单的情况：你的同事刚把他写的几个提交 push 到了中央仓库，你怎么把它们取下来。

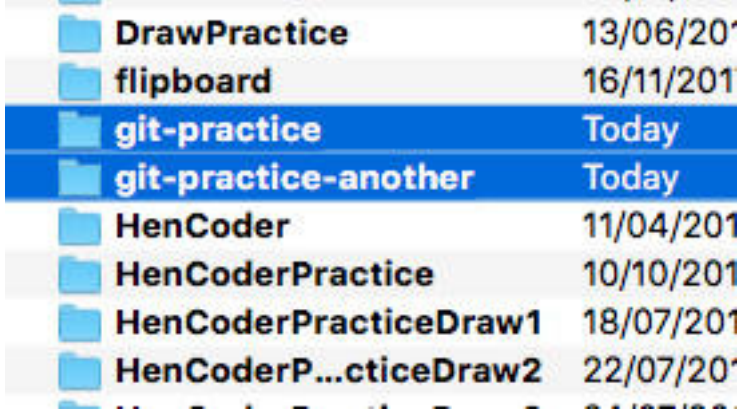
### 假装同事

除非你能要求同事陪你练习 Git，否则，你只能一人分饰两角：时不时地模拟一下同事的操作，来达到独自一人做多人项目练习的效果。而为了模拟同事的操作，你需要在你的 `/git-practice` 目录旁再 clone 一次中央仓库到本地，来当做你的模拟同事的本地仓库：

```
git clone https://github.com/rengwuxian/git-practice.git git-practice-anoth
```

> 注意！执行这条之前别忘了先 `cd ..` 切换到父目录，不然你会把新的本地仓库创建在原先的本地仓库的工作目录内。

为了目录名称不冲突，这次的 `clone` 需要加一个额外参数（上面那行代码最后的 `git-practice-another`）来手动指定本地仓库的根目录名称。在这条指令执行完成后，你就有了两个内容相同的本地仓库：




你的 git-practice 和「同事」的 git-practice-another

现在，你就可以假装 `/git-practice` 这个目录是你的电脑上的工作目录，而那个新建的 `/git-practice-another` 是你同事的电脑上的工作目录。

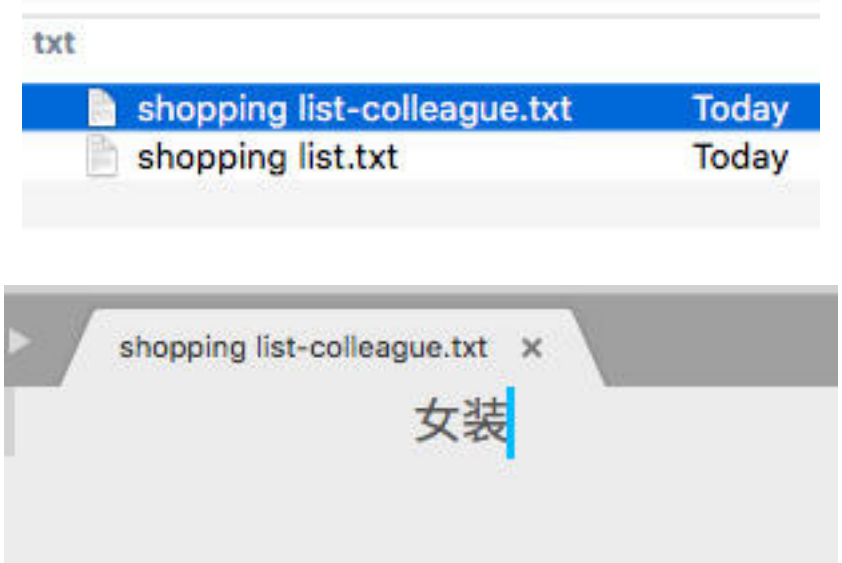
如果你对这样的操作有担心，大可不必，这种操作不会有任何问题。因为 Git 的管理是目录级别，而不是设备级别的。也就是说，`/git-practice` 目录内的 `.git` 只管理 `/git-practice` 里的内容，`/git-practice-another` 目录内的 `.git` 也只管理 `/git-practice-another` 里的内容，它们之间互不知晓，也互不影响。

## 帮同事提交代码并 push 到中央仓库

要在你的电脑上把同事的代码取下来，首先 GitHub 上要有你同事的新代码，也就是说，你首先要把同事的代码 push 到中央仓库去。所以第一步，切换到同事的目录：

```
cd git-practice-another
```

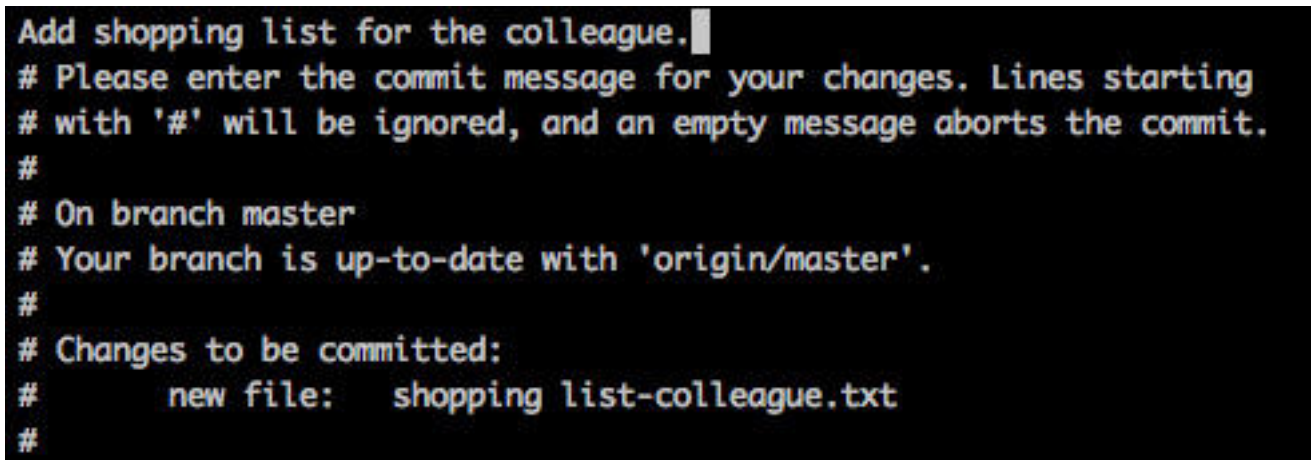
现在，到了同事的工作目录，你的身份就是你的同事了。帮他写点代码吧！嗯……写点什么好呢？




嗯，「女装」

嗯，看起来不错，帮他提交吧：

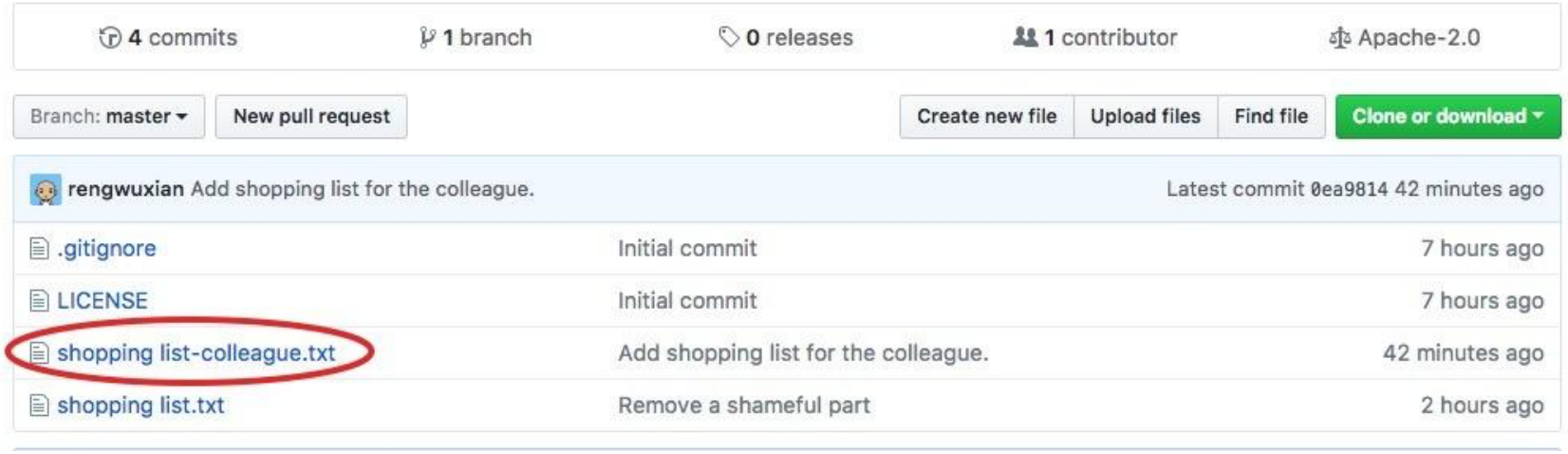
```
git add shopping\ list-colleague.txt
git commit
```




记得填写提交信息

提交完成以后，`push` 到 GitHub：

```
git push
```




GitHub 上已经能看到刚更新的文件

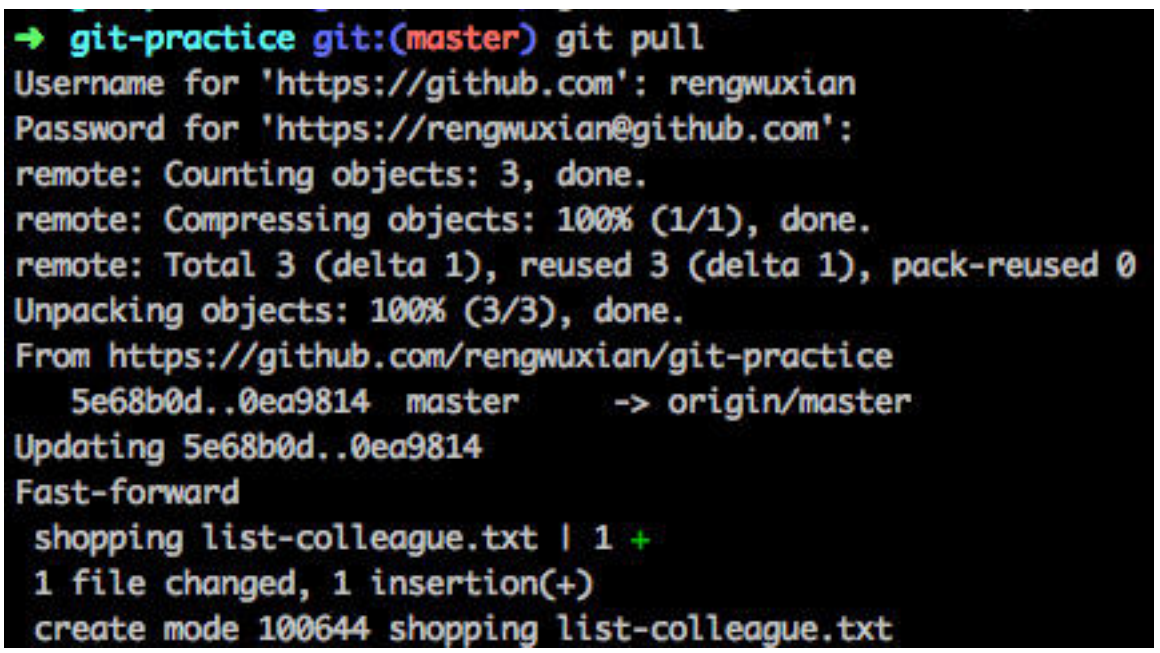
## 把同事 push 的新代码取下来

GitHub 上有了同事的新代码以后，你就可以变回自己，把「他」刚 push 上去的代码取到你的仓库了。首先，切回 `/git-practice` 目录：

```
cd ../git-practice
```

然后，把同事的代码取下来吧！从远程仓库更新内容，用的是一个新的指令：`pull`。

```
git pull
```




用 git pull 把代码从远程仓库取下来

这时候再看看你的本地目录，可以看到，同事提交的 `shopping list-colleagure.txt` 已经同步到本地了。也就是说，现在你的本地仓库和同事

的又一致了。

## 多人合作的基本工作模型

这就完成了一次简单的合作流程：

1. 同事 commit 代码到他的本地，并 push 到 GitHub 中央仓库
2. 你把 GitHub 的新提交通过 pull 指令来取到你的本地

通过这个流程，你和同事就可以简单地合作了：你写了代码，commit，push 到 GitHub，然后他 pull 到他的本地；他再写代码，commit, push 到 GitHub，然后你再 pull 到你的本地。你来我往，配合得不亦乐乎。

但是，这种合作有一个严重的问题：同一时间内，只能有一个人在工作。你和同事其中一个人写代码的时候，另一个人不能做事，必须等着他把工作做完，代码 push 到 GitHub 以后，自己才能把 push 上去的代码 pull 到自己的本地。而如果同时做事，就会发生冲突：当一个人先于另一个人 push 代码（这种情况必然会发生），那么后 push 的这个人就会由于中央仓库上含有本地没有的提交而导致 push 失败。

> 为什么会失败?
>
> 因为 Git 的push 其实是用本地仓库的 commits 记录去覆盖远端仓库的 commits 记录（注：这是简化概念后的说法，push 的实质和这个说法略有不同），而如果在远端仓库含有本地没有的 commits 的时候，push （如果成功）将会导致远端的 commits 被擦掉。这种结果当然是不可行的，因此 Git 会在 push 的时候进行检查，如果出现这样的情况，push 就会失败。

怎么办？下面就说。

## push 发生冲突

在现实的团队开发中，全队是同时并行开发的，所以必然会出现当一人 push 代码时，中央仓库已经被其他同事先一步 push 了的情况。为了不让文段显得太过混乱，这里我就不带着你一步步模拟这个过程了。如果你希

望模拟的话，这里是步骤：

1. 切到 `git-practice-another` 去，假扮成你的同事做一个 `commit`，然后 `push` 到 GitHub
2. 切回 `git-practice` 变回你自己，做一个不一样的 `commit`。

这个时候，远端中央仓库已经有了别人 `push` 的 `commit`，现在你如果 `push` 的话：

```
git push
```

```
➜ git-practice git:(master) git push
To https://github.com/rengwuxian/git-practice.git
 ! [rejected]        master -> master (fetch first)
error: failed to push some refs to 'https://github.com/rengwuxian/git-practice.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

push 被拒绝

从上图中的提示语可以看出来（好吧这么长的提示语有人表示不想看），由于 GitHub 的远端仓库上含有本地仓库没有的内容，所以这次 `push` 被拒绝了。这种冲突的解决方式其实很简单：**先用 `pull` 把远端仓库上的新内容取回到本地和本地合并，然后再把合并后的本地仓库向远端仓库推送。**

```
git pull
```

```
Merge branch 'master' of https://github.com/rengwuxian/git-practice

# Please enter a commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
```

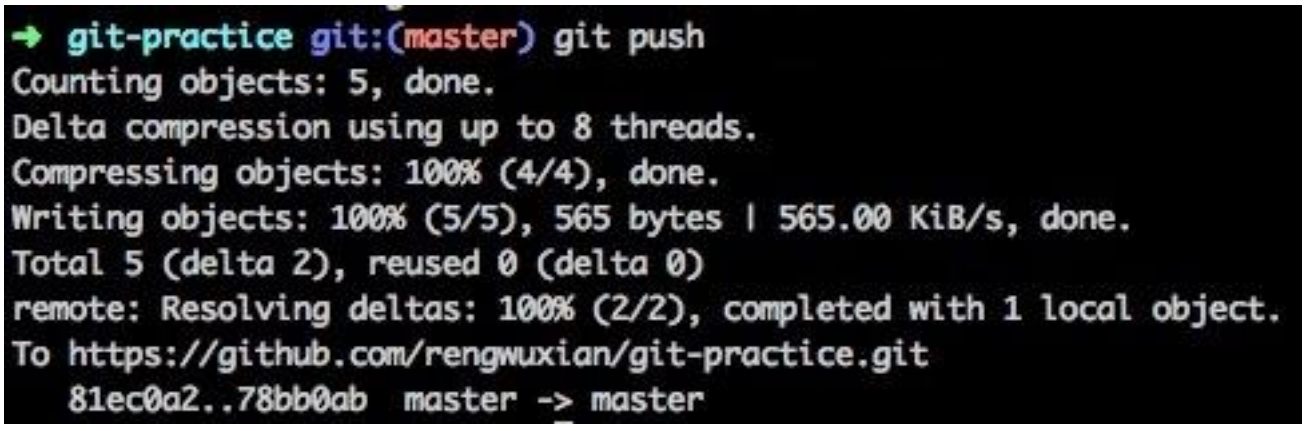
在上面的文段里，我已经举过一次 `git pull` 的例子，但这次的 `git pull` 操作并没有像之前的那样直接结束，而是进入了上图这样的一个输入提交

信息的界面。这是因为当 pull 操作发现不仅远端仓库包含本地没有的 commits，而且本地仓库也包含远端没有的 commits 时，它就会把远端和本地的独有 commits 进行合并，自动生成一个新的 commit ，而上图的这个界面，就是这个自动生成的 commit 的提交信息界面。另外，和手动的 commit 不同，这种 commit 会自动填入一个默认的提交信息，简单说明了这条 commit 的来由。你可以直接退出界面来使用这个自动填写的提交信息，也可以修改它来填入自己提交信息。

> 这种「把不同的内容进行合并，生成新的提交」的操作，叫做合并（呵呵呵哈哈），它所对应的 Git 指令是 merge。事实上，git pull 这个指令的内部实现就是把远程仓库使用 git fetch 取下来以后再进行 merge 操作的。关于更多 merge 的介绍，我会在后面说，这节先不讲了。

在退出提交信息的界面后，这次 pull 就完成了：远端仓库被取到了本地，并和本地仓库进行了合并。在这个时候，就可以再 push 一次了。由于现在本地仓库已经包含了所有远端仓库的 commits，所以这次 push 不会再失败：

```
git push
```




push 成功

这样，就把 push 时本地仓库和远端仓库内容冲突的问题解决了。

## 小结：多人合作的基本工作模型 2.0

这样，就把刚才的那个「多人合作的基本工作模型」进行了改良：

1. 写完所有的 `commit` 后，不用考虑中央仓库是否有新的提交，直接 `push` 就好
2. 如果 `push` 失败，就用 `pull` 把本地仓库的提交和中央仓库的提交进行合并，然后再 `push` 一次

到此为止，这个工作模型已经是一个最简单的可用的工作模型了。一个小团队如果对版本管理没有什么要求的话，这个工作模型已经可以让团队用来合作开发了。

但是，这种模型由于过于简单，所以只能适用于对代码管理没太大需求的开发团队。接下来，我将介绍一种基于 branch 的工作模型，很多中小型团队的代码开发都是采用的这种工作模型。不过.....需要花 4 节的时间，因为它涉及到了 Git 的一些核心理论，一些许多 Git 老手都没有真正掌握的理论。